UNIT – IV: Unsupervised Learning

Clustering basics (Partitioned, Hierarchical and Density based) -Means clustering – K- Mode clustering – Self organizing maps – Expectation maximization – Principal Component Analysis – Kernel PCA – tSNE (t-distributed stochastic neighbour embedding) - Metrics & Error Correction.

Clustering is a type of unsupervised machine learning technique where data is grouped into clusters based on similarity. The goal is to find inherent patterns or structures within data. Clustering methods can be broadly categorized into three types: **Partitioned Clustering**, **Hierarchical Clustering**, and **Density-based Clustering**.

1. Partitioned Clustering In partitioned clustering, the data is divided into a set of distinct, non-overlapping clusters. Each data point is assigned to exactly one cluster.

- **K-means clustering** is the most popular method under partitioned clustering.
  Process:
  1. Initialize K centroids randomly.
  2. Assign each data point to the nearest centroid based on Euclidean distance.
  3. Update the centroids as the mean of points in each cluster.
  4. Repeat until centroids stabilize.

  Key Features: Fast and efficient but sensitive to outliers.

- **K-medoids Clustering**

  Process:

  1. Initialize K medoids (actual data points).
  2. Assign each data point to the nearest medoid.
  3. Update medoids by selecting the point that minimizes total distance within the cluster.
  4. Repeat until medoids stabilize.

  Key Features: More robust to outliers than K-means.

- **K-modes Clustering**

  Process:

  1. Initialize K modes (categorical prototypes).
  2. Assign each data point to the nearest mode based on dissimilarity (number of mismatches).
  3. Update modes by selecting the most frequent category for each attribute.
  4. Repeat until modes stabilize.

  Key Features: Designed for clustering categorical data effectively.

2. Hierarchical Clustering

Hierarchical clustering builds a tree-like structure (dendrogram) that groups data based on their similarity. This method does not require the number of clusters to be defined upfront.

There are two types of hierarchical clustering:

- **Agglomerative (Bottom-up approach):**
  - Starts by treating each data point as a separate cluster.
  - It then iteratively merges the two most similar clusters until only one cluster remains.
- **Divisive (Top-down approach):**
  - Starts with all data points in a single cluster and recursively splits the most dissimilar clusters.

The similarity between clusters can be calculated using different linkage methods (e.g., single-linkage, complete-linkage, average-linkage).

3. Density-Based Clustering

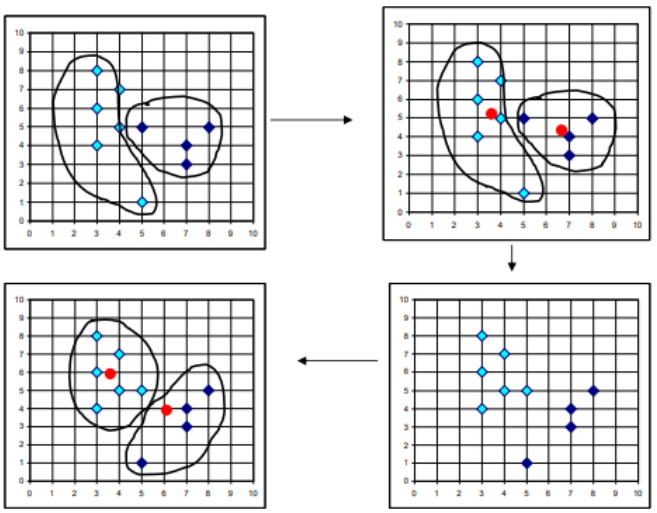
Density-based clustering identifies clusters based on the density of points in the data space. It is well-suited for identifying clusters of arbitrary shape and handling outliers effectively.

- **DBSCAN (Density-Based Spatial Clustering of Applications with Noise)** is the most popular algorithm in this category.
  - It groups together closely packed points (points that are close to each other) and marks points that are in low-density regions as outliers.
  - The algorithm requires two parameters: the **epsilon** (maximum distance between points in a cluster) and **minPts** (the minimum number of points required to form a dense region).
- **OPTICS (Ordering Points to Identify the Clustering Structure)** is another density-based algorithm that works similarly to DBSCAN but produces a more detailed view of the cluster structure.

K-means Clustering Algorithm

K-means clustering is a partitioned clustering technique that groups numerical data into **K clusters**. It minimizes a **clustering cost function** (sum of squared distances) to form well-defined clusters.

K-means uses a similarity measure (typically **Euclidean distance**) and iteratively adjusts cluster centers (centroids) to minimize the total variance within clusters.



Steps in the K-means Algorithm

Let:

- X={x1,x2,…,xn} be the set of data points.
- V={v1,v2,…,vk} be the set of K cluster centers.

1. **Initialize Cluster Centers**
   - Randomly select **K** initial cluster centers (centroids).
2. **Assign Points to Nearest Cluster**
   - For each data point xi, calculate its distance to each cluster center vj
   - Assign xi to the cluster with the nearest center based on the minimum distance.
3. **Update Cluster Centers**

Recalculate the cluster centroids using the formula:

$$v_j = \frac{1}{c_j} \sum_{x_i \in C_j} x_i$$

where C_j is the set of data points in cluster j, and c_j is the number of points in that cluster.

4. **Recalculate Distances**
   - Compute the distance between each data point and the updated cluster centers.
5. **Check for Convergence**
   - If no data point has changed its cluster assignment, stop.
   - Otherwise, repeat from **Step 2**.

Key Points

- **Objective**: Minimize the sum of squared distances between points and their assigned cluster centroids.
- **Complexity**: Iterative algorithm with time complexity O(n×k×d), where n is the number of points, k is the number of clusters, and d is the dimensionality.

- **Convergence**: Guaranteed to converge, but may reach a local minimum depending on the initial cluster centers.

Advantages

- Simple and efficient for large datasets.
- Easy to implement and interpret.

Disadvantages

- Sensitive to the choice of initial centroids.
- Requires specifying the number of clusters (K) in advance.
- Assumes clusters are spherical and evenly sized.

PROBLEMS:

1)The data mining task is to cluster points into three clusters, where the points are A1(2,10), A2(2,5), A3(8,4), B1(5,8), B2(7,5), B3(6,4) , C1(1,2), C2(4,9) The distance function is Manhattan distance. Suppose initially we assign A1 ,B1,C1 as the center of each cluster, use the k-means algorithm to show only a) The three clusters after the first round of execution . b) The final three clusters

https://www.youtube.com/watch?v=KzJORp8bgqs&list=PL4gu8xQu0_5KiYnRlueicckEmpFAiRD5Y&index=5

2)https://www.youtube.com/watch?v=nWmeC61DgBo&list=PL4gu8xQu0_5KiYnRlueicckEmpFAiRD5Y&index=8
L1 dist(Manhattan dist):

For two points $P = (x_1, y_1)$ and $Q = (x_2, y_2)$, the Manhattan distance d is given by:

$$d(P, Q) = |x_1 - x_2| + |y_1 - y_2|$$

3)https://www.youtube.com/watch?v=FGXkbawTHRQ&list=PL4gu8xQu0_5KiYnRlueicckEmpFAiRD5Y&index=27Selecting the optimal number of clusters (**K**) is crucial in clustering algorithms like K-means. Two commonly used methods are:

1. Elbow Method

The Elbow Method helps find the optimal number of clusters by analyzing the **Within-Cluster Sum of Squares (WCSS)** or **Inertia**. The plot forms an elbow-like shape, and the point where the curve bends is the optimal number of clusters.

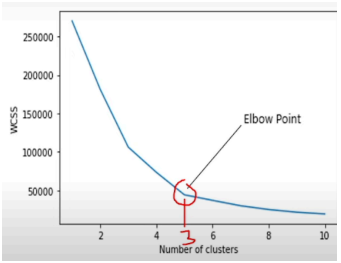
Pros: Simple to understand and apply.

Cons: The elbow point is sometimes subjective and not always clear.

Steps:

1. Run K-means clustering for a range of values of K (e.g., 1 to 10).
2. For each K, calculate the WCSS:

$$WCSS = \sum_{i=1}^{K} \sum_{x \in C_i} \|x - v_i\|^2$$



where C_i is the cluster, and v_i is the centroid of cluster i.

3. Plot the WCSS against K.
4. Identify the "elbow point" where the rate of decrease sharply slows down. This is the optimal K.

2. Silhouette Method

The Silhouette Method evaluates the quality of clusters by measuring how similar data points are to their own cluster compared to other clusters.

Steps:

1. For each data point i, calculate:
   - **a(i)**: Average distance to all points in the same cluster.
   - **b(i)**: Average distance to all points in the nearest neighboring cluster.
2. Compute the **Silhouette Coefficient** for each point:

$$S(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

   The coefficient ranges from -1 to 1:
   - **1**: Perfect clustering.
   - **0**: Overlapping clusters.
   - **Negative**: Wrong clustering.
3. Calculate the average silhouette score for all points for different K values.
4. Choose the K with the highest average silhouette score.

Pros: Provides a clear measure of cluster quality.

Cons: More computationally intensive than the Elbow Method.

K-modes Algorithm (For Categorical Data) To cluster categorical data by minimizing dissimilarity. **Steps:**

1. **Initialize Modes**
   - Arbitrarily select k objects as initial cluster modes.
2. **Assign Objects to Clusters**
   - Assign each object to the cluster with the most similar mode based on the least number of mismatches (dissimilarity).

3. **Update Modes**
    - For each cluster, update the mode by finding the **most frequently occurring value** for each attribute (column).
4. **Repeat Until Convergence**
    - Repeat steps 2 and 3 until the clusters remain unchanged in two consecutive iterations.

k-modes **algorithm problem**

Objects	Income	Age	STUDENT
1	Low	Middle	No
2	High	Senior	No
3	Low	Youth	Yes
4	Medium	Youth	Yes
5	High	Middle	No
6	High	Youth	No

Step1 : select 2 objects from the 6 objects as the initial modes
(i.e., we have choosen O1, O4 as intial model)
So our modes are (Low, Middle, No) and (Medium, Youth, Yes)
Step 2: Assign the objects to the cluster based on the similarity between the mode and object.
Ex for finding the distance between (low,medium,no) and (high ,senior, no)=
(low-high)+(medium-senior)+(no-no) = (1+1-0)=2
In the same way find the distance between the objects and modes.

k-modes **algorithm problem**

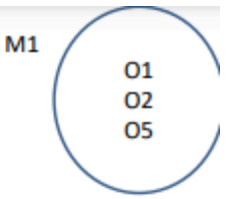
Objects Modes	01	02	03	04	05	06
M1=01	**0**	**2**	2	3	**1**	2
M2=04	3	3	**1**	**0**	3	**2**

Step 2: Assign the objects to the cluster based on the similarity between the mode and object.



Step 3: find the mode after clustering all the objects is step 2.
In the cluster 1 we have O1,O2,O5

Objects	Income	Age	STUDENT
1	Low	Middle	No
2	High	Senior	No
5	High	Middle	No
New mode 1	High	Middle	No





In the cluster 2 we have O3,O4,O6

Objects	Income	Age	STUDENT
3	Low	Youth	Yes
4	Medium	Youth	Yes
6	High	Youth	No
New mode 2	High	Youth	yes

k-modes **algorithm problem**

Step 2: re Assign the objects to the cluster based on the similarity between the mode and object.

Objects Modes	01	02	03	04	05	06
M1	**1**	**2**	3	3	**0**	2
M2	3	2	**1**	**1**	2	**1**



Step 4: the clusters are same in the consecutive iterations So halt

DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

Purpose: DBSCAN clusters data points based on density, identifying core points, noise, and clusters without needing to specify the number of clusters in advance.**Key Concepts:**

1. **Neighborhood (ε\varepsilonε)**: Objects within a radius ε\varepsilonε of a point.
2. **Core Object**: A point with at least **minPts** neighbors within ε\varepsilonε.
3. **Directly Density Reachable**: Points within ε\varepsilonε radius of a core point.
4. **Density Reachable**: Points reachable through a chain of core points.

Steps in DBSCAN Algorithm:

1. **Select Random Object**
    - Choose an unclustered point randomly.
2. **Check Core Object**
    - If the point has at least **minPts** neighbors within ε\varepsilonε, it is a **core object**.
    - If not, mark it as **noise** and go back to Step 1.
3. **Expand Cluster**
    - Add all **directly density reachable** points to a candidate set.
4. **Process Candidate Set**
    - Remove a point from the candidate set and repeat Step 2 for it.
    - Continue until the candidate set is empty.
5. **Form Cluster**
    - Group all **density reachable** points together into one cluster.
6. **Repeat**
    - Continue until all points are either clustered or marked as noise.

https://www.youtube.com/watch?v=-p354tQsKrs&list=PL4gu8xQu0_5KiYnRlueicckEmpFAiRD5Y&index=9
https://www.youtube.com/watch?v=ZOLYaa9Jex0&list=PL4gu8xQu0_5KiYnRlueicckEmpFAiRD5Y&index=10

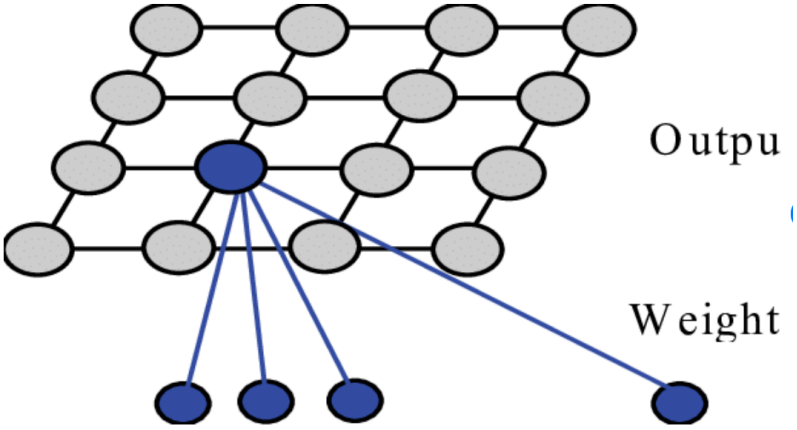
Advantages: Can detect clusters of arbitrary shape,Automatically identifies noise (outliers),No need to specify the number of clusters in advance.

Disadvantages: Sensitive to the choice of ε\varepsilonε and **minPts**.Not suitable for datasets with varying densities.

Self organizing maps

Self-Organizing Maps (SOMs), also known as **Kohonen Maps**, are a type of **artificial neural network** used for unsupervised learning. They are mainly used for **dimensionality reduction** and **visualization** of high-dimensional data.

SOMs map high-dimensional input data onto a **2D grid** (usually) of neurons. Each neuron in the grid has a weight vector of the same dimension as the input data, and the goal is to train the network such that similar data points in the input space are mapped close to each other on the grid.



Kohonen Self-Organizing Map

Advantages of SOMs:

- Effective for visualizing high-dimensional data.
- Can handle both numerical and categorical data.
- Good for discovering hidden patterns or structures in data.

Disadvantages of SOMs:

- Sensitive to initialization and parameters (like learning rate).
- Computationally expensive for large datasets.
- May require a lot of iterations to converge.

Applications of SOMs:

- Data Visualization
- Clustering
- Feature Mapping
- Pattern Recognition

https://www.youtube.com/watch?v=InVvyioWDIw&list=PL4gu8xQu0_5JK6KmQi-Qx5hI3W13RJbDY&index=24

EM Algorithm (Expectation-Maximization Algorithm)

The **Expectation-Maximization (EM)** algorithm is a powerful statistical method used for parameter estimation in models with **latent variables** (hidden or unobserved variables). It is commonly used for clustering, particularly in **Gaussian Mixture Models (GMMs)**, and other probabilistic models.

The EM algorithm iteratively estimates the parameters of the model by alternating between two steps: **Expectation (E-step)** and **Maximization (M-step)**.

Steps in the EM Algorithm:

1. **Initialization**
    - Start with initial guesses for the parameters of the model (e.g., means, variances, and mixing coefficients in a GMM).
2. **E-step (Expectation Step)**
    - Given the current parameter estimates, calculate the **expected value** of the latent (hidden) variables based on the observed data.
    - This step computes the probability of the data belonging to different clusters (or distributions) using the current parameters.
3. **M-step (Maximization Step)**
    - Update the parameters of the model by maximizing the likelihood function (or minimizing the negative log-likelihood) based on the expected values calculated in the E-step.
4. **Repeat**
    - Repeat the E-step and M-step until the parameters converge (i.e., the change in the parameter values between iterations is below a threshold).

Applications of the EM Algorithm:

- Clustering
- Missing Data
- Image and Speech Recognition
- Hidden Markov Models (HMMs)

Advantages of the EM Algorithm:

- Can handle models with hidden variables.
- Flexible and widely applicable in various probabilistic models.
- Often finds a good solution even with complex, high-dimensional data.

Disadvantages of the EM Algorithm:

- **Local Optima**: It is sensitive to the initial parameter estimates and may converge to a local optimum.
- **Computational Complexity**: Can be computationally expensive, especially for large datasets and models with many parameters.
- **Convergence Speed**: The algorithm may require many iterations to converge.

Example:

You have a bag with candies of different colors (e.g., Red, Green, and Blue), but you don't know the exact proportion of each color. You use the EM algorithm to estimate these proportions.

Step 1: Expectation Step (E Step)

1. You take a candy out of the bag without looking at its color.
2. A friend guesses the color of the candy based on prior knowledge, providing both a guess and their confidence level for each color.
3. For example:
    a. First candy: 80% chance Red, 10% Green, 10% Blue.
    b. Second candy: 30% Red, 60% Green, 10% Blue.
    c. Third candy: 20% Red, 10% Green, 70% Blue.

Step 2: Maximization Step (M Step)

You use your friend's guesses to estimate the number of candies of each color.

Sum the confidence values for each color across all guesses:

Red: (0.8+0.3+0.2)/3=0.43

(0.8+0.3+0.2)/3=0.43 (43%)

Green: (0.1+0.6+0.1)/3=0.27

(0.1+0.6+0.1)/3=0.27 (27%)

Blue: (0.1+0.1+0.7)/3=0.30

(0.1+0.1+0.7)/3=0.30 (30%)

Update your estimates of the color proportions in the bag based on these calculations.

Step 3: Repeat

- Go back to the E-step and continue the process with updated guesses and estimates.
- Over multiple iterations, the guesses and estimates converge to the actual proportions of candies in the bag.

AGGLOMERATIVE CLUSTERING:

Linkage refers to the methods used to determine the distance between clusters in hierarchical clustering algorithms. These methods define how to compute the "distance" between two clusters based on the individual distances between their members.

1. Single Linkage: In **single linkage** clustering, the distance between two clusters is defined as the **minimum distance** between any single pair of points in the two clusters.

Explanation: If you have two clusters A and B, the distance between them is the shortest distance between any point in A and any point in B.

Mathematically:

$$D_{\text{single}}(A, B) = \min\left(\text{dist}(a_i, b_j)\right) \quad \forall\, a_i \in A,\, b_j \in B$$

Where a_i and b_j are data points in clusters A and B, and $\text{dist}(a_i, b_j)$ is the distance between them.

Advantages:

- Can form elongated or irregularly shaped clusters.
- Simple to compute.

Disadvantages:

- Sensitive to noise and outliers (since a single pair of points can influence the distance calculation).
- May result in **chaining**, where clusters grow by connecting distant points.

https://www.youtube.com/watch?v=tXYAdGn-SuM

2. Complete Linkage: In **complete linkage** clustering, the distance between two clusters is defined as the **maximum distance** between any pair of points in the two clusters.

Explanation: For clusters A and B, the distance between them is the largest distance between any point in A and any point in B.

Mathematically:

$$D_{\text{complete}}(A, B) = \max\left(\text{dist}(a_i, b_j)\right) \quad \forall\, a_i \in A,\, b_j \in B$$

Advantages:Tends to produce compact, spherical clusters.,Less sensitive to noise and outliers compared to single linkage.

Disadvantages:May be less flexible in terms of handling clusters of different shapes.,Can result in a more rigid hierarchical structure.

https://www.youtube.com/watch?v=0A0wtto9wHU&t=25s

Principal Component Analysis

Principal Component Analysis (PCA) is a **dimensionality reduction** technique used to reduce the complexity of data while preserving as much information as possible. It transforms data into a new coordinate system where the axes (called **principal components**) are ordered by the variance of the data along them. PCA is commonly used for feature extraction, noise reduction, and visualization.

How PCA Works:

1. **Standardize the Data** (Optional but recommended):
    - If the data features have different scales, it's important to standardize them (i.e., subtract the mean and divide by the standard deviation) to bring them to the same scale.
2. **Compute the Covariance Matrix**:
    - Calculate the covariance matrix to understand how the features vary with respect to each other. The covariance matrix captures the relationships between different features.
3. **Compute the Eigenvalues and Eigenvectors**:
    - Calculate the eigenvalues and eigenvectors of the covariance matrix. Eigenvectors represent the directions of maximum variance (the principal components), and eigenvalues represent the magnitude of the variance along those directions.
4. **Sort Eigenvalues and Eigenvectors**:
    - Sort the eigenvectors in descending order of their corresponding eigenvalues. This gives the **principal components** in order of importance (most to least variance explained).
5. **Select the Top k Principal Components**:
    - Choose the top k eigenvectors (based on the highest eigenvalues) to form a new feature space. The number of principal components (k) to retain depends on the desired amount of variance to capture.
6. **Transform the Data**:
    - Project the original data onto the selected principal components to obtain the reduced-dimensional representation of the data.

Mathematical Formulation:

1. Given a dataset X with n data points and d dimensions, we want to reduce the data to a lower k-dimensional space.
2. The **covariance matrix** C is computed as

: $$C = \frac{1}{n} X^T X$$

3. Compute the **eigenvalues** λ and **eigenvectors** v of the covariance matrix.

4. Sort the eigenvalues and select the eigenvectors corresponding to the largest eigenvalues.
5. The reduced data is obtained by projecting the original data onto the selected eigenvectors.

Applications of PCA:

- **Dimensionality Reduction**: PCA is widely used to reduce the number of features in high-dimensional data, which can improve the performance of machine learning algorithms.
- **Data Visualization**: By reducing the dimensions to 2 or 3, PCA helps visualize high-dimensional data (e.g., in 2D or 3D scatter plots).
- **Noise Reduction**
- **Feature Engineering**

Advantages of PCA:

- **Improved Efficiency**: Reduces the number of features, leading to faster training of machine learning models.
- **Noise Reduction**: Eliminates less important features that could introduce noise.
- **Interpretability**: Helps in understanding the structure of the data by identifying the directions of maximum variance.

Disadvantages of PCA:

- **Loss of Information**: Some information may be lost if too many principal components are discarded.
- **Linear Assumption**: PCA assumes that the data lies on a linear subspace, which may not always be true.
- **Hard to Interpret**: Principal components are combinations of original features, which can be difficult to interpret.

Ex :https://www.youtube.com/watch?v=ZtS6sQUAh0c

Kernel PCA

Kernel Principal Component Analysis (Kernel PCA) is an extension of PCA that allows non-linear dimensionality reduction by mapping data into a higher-dimensional space using kernel functions (like RBF, polynomial) before applying PCA. It is useful for data with non-linear relationships, where traditional PCA fails.

Steps in Kernel PCA:

1. **Kernel Mapping**: Use a kernel function (e.g., RBF) to map the data to a higher-dimensional space without explicitly transforming the data.
2. **Compute the Kernel Matrix**: Calculate the pairwise kernel values for all data points.
3. **Center the Kernel Matrix**: Subtract the mean row and column from the kernel matrix.

4. **Eigenvalue Decomposition**: Perform eigenvalue decomposition to find the principal components in the feature space.
5. **Projection**: Project the data onto the selected principal components to reduce dimensionality.

Advantages:

- Can capture non-linear relationships in the data.
- Flexible with different kernel functions.
- Useful for complex data patterns.

Disadvantages:

- Computationally expensive.
- Choice of kernel function can affect performance.
- Results may be hard to interpret in the high-dimensional space.

Applications:

- Non-linear dimensionality reduction.
- Pattern recognition and machine learning preprocessing.
- Visualizing complex, high-dimensional data.

tSNE (t-distributed stochastic neighbour embedding)

t-SNE is a powerful non-linear dimensionality reduction technique primarily used for the visualization of high-dimensional data in two or three dimensions. It is particularly effective for visualizing clusters or patterns in complex datasets, such as those arising in machine learning or bioinformatics.

How t-SNE Works:

- **Similarity Measurement (High-dimensional space):** t-SNE starts by computing pairwise similarities between points in the **high-dimensional space**. It does this using **conditional probabilities**: the probability that a data point xj is a neighbor of point xi, which is modeled using a Gaussian distribution. The similarity between two points is based on the **Euclidean distance** between them.

$$P_{ij} = \frac{\exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right)}{\sum_{k \neq i} \exp\left(-\frac{\|x_i - x_k\|^2}{2\sigma^2}\right)}$$

where Pij is the probability of point xj being a neighbor of point xi, and σ is a parameter that controls the width of the Gaussian.

- **Similarity Measurement (Low-dimensional space):**t-SNE then tries to embed the points into a **low-dimensional space** (typically 2D or 3D), where the similarities are modeled using a **t-distribution** with one degree of freedom (Cauchy distribution), rather than a Gaussian distribution. The use of a t-distribution helps to **separate clusters** better and reduces crowding problems.

$$Q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq i}(1 + \|y_i - y_k\|^2)^{-1}}$$

where Qij represents the similarity between points yi and yj in the low-dimensional space.

- **Minimizing the Divergence (KL Divergence):** The algorithm minimizes the **Kullback-Leibler (KL) divergence** between the probability distributions in the high-dimensional space (from step 1) and the low-dimensional space (from step 2). The objective is to make the pairwise similarities in the low-dimensional space as similar as possible to those in the original space.

$$\mathrm{KL}(P||Q) = \sum_{i \neq j} P_{ij} \log \frac{P_{ij}}{Q_{ij}}$$

- **Optimization:**t-SNE uses **gradient descent** to iteratively adjust the positions of points in the low-dimensional space to minimize the KL divergence.

Key Features:

- **Preserves Local Structure**: Keeps similar points close in the low-dimensional space.
- **Non-linear Mapping**: Captures complex, non-linear relationships, unlike PCA.
- **Emphasizes Clusters**: Effective at visualizing clusters and patterns in high-dimensional data.

Advantages:

- **Effective Visualization**: Produces clear 2D/3D visualizations of patterns, clusters, and outliers.
- **Non-linear**: Captures non-linear relationships better than linear methods like PCA.
- **Preserves Local Structure**: Reveals similarities and structures in the data.

Disadvantages:

- **Computationally Expensive**: Slow and memory-intensive, especially for large datasets.
- **Non-deterministic**: Results can vary across runs.
- **Distorts Global Structure**: Doesn't preserve the overall distances between clusters.
- **Hard to Interpret**: Limited to 2D/3D visualization, which may not fully capture data complexity.

Applications: Data Visualization,Clustering

Metrics and Error Correction in Unsupervised Learning

Internal Metrics (No ground truth required):

- **Silhouette Score**: Measures how well-separated clusters are. Higher values indicate better clustering.
- **Davies-Bouldin Index**: Measures cluster similarity. Lower values indicate better clustering.
- **Dunn Index**: Measures the compactness and separation of clusters. Higher values are better.
- **Inertia**: Measures the compactness of clusters. Lower inertia is better.

External Metrics (Requires ground truth):

- **Rand Index**: Measures similarity between two clusterings. Higher values indicate better agreement with the ground truth.
- **Adjusted Rand Index (ARI)**: Corrects for chance, giving a score between -1 and 1.
- **Fowlkes-Mallows Index (FMI)**: Measures precision and recall for clusters. Higher values are better.

Error Correction in Clustering:

- **Model Re-Initialization**: Re-initializing centroids to avoid local minima.
- **Post-Processing**: Merging/splitting clusters and removing noise to improve clustering.
- **Feature Selection/Extraction**: Reducing dimensionality or selecting relevant features to reduce noise.
- **Consensus Clustering**: Combining multiple clustering results to improve stability.
- **Post-Clustering Adjustment**: Using algorithms like EM for refining clusters.
- **Supervised Refinement**: Using labeled data to adjust clusters.